TransferIa · Documentos de cambios

Punto de partida: barra de progreso e hilos

Entrega 1 · El programa original y la CLI con barra y varios hilos

Commits: `5f86333`, `1f065f1`, `85ea6f0`

Fechas: 20 y 21 de agosto de 2026

Documento generado: 27 de octubre de 2026

Pruebas: caras generadas por IA, en CPU

Contenido

1. Resumen de la entrega 2
2. Cómo funciona TransferIa por dentro 2
2.1 Las piezas del proyecto original 3
2.2 La ventana original 3
2.3 La CLI original, probada 4
3. El problema: procesar un video sin saber cuánto falta 5
4. La solución: barra de progreso y varios hilos 6
4.1 La barra de progreso 6
4.2 Varios hilos, pero en orden 6
5. Cómo se comprobó 7
6. "mejora normal": la opción --device 8
7. Cómo lo pruebas tú 8
8. Lo que aprendimos 8
Glosario 8

Si el índice aparece vacío, en Word haz clic derecho sobre él → Actualizar campos.

1. Resumen de la entrega

Este documento es el **punto de partida** de la historia de TransferIa. Explica cómo era el programa original (commit `5f86333`, 20 de agosto) y los dos commits que lo siguieron:

Commit	Fecha	Qué trajo	Qué gana el usuario
`5f86333` trasnfer ia cli	20 ago	El programa original: ventana, CLI y modelos	Cambiar una cara en una imagen o un video
`1f065f1` agregando barra y cli con multihilos	20 ago	Barra de progreso y varios hilos en la CLI	Ver cuánto falta mientras procesa un video
`85ea6f0` mejora normal	21 ago	Documentación de `--device` en el README	Nada todavía: la opción no se programó (ver sección 6)

> **Importante:** Estos tres commits los hiciste tú. Este documento explica *qué* cambió y *cómo funciona* leyendo el código y ejecutándolo; el *por qué* original no quedó registrado, así que donde hace falta se dice "probablemente".

2. Cómo funciona TransferIa por dentro

Antes de ver los cambios, conviene entender el recorrido de una imagen. La idea es simple: **se busca la cara en las dos imágenes, se calcula "quién es" la persona de la fuente y se dibuja esa identidad encima de la cara del destino**, respetando su postura y su gesto.

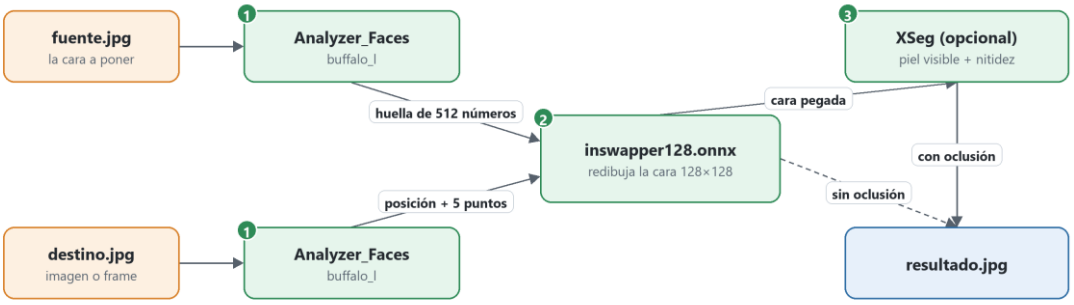


Figura 1. *Recorrido de una imagen: detectar (1), cambiar la cara (2) y, si se pide, oclusión (3)*

2.1 Las piezas del proyecto original

Archivo	Responsabilidad
`app.py` (755 líneas)	La ventana **Transfer-IA Studio** (Tkinter): arrastrar fuente y destino, vista previa, opciones y botón de procesar.
`cli.py` (234 líneas)	La misma tarea desde la consola: `python cli.py process --source … --target … --output …`
`core/Face_Analyzer.py`	Envuelve `FaceAnalysis` de InsightFace (modelo **buffalo_l**): detecta caras y las ordena de izquierda a derecha.
`core/Face_Swaper.py`	Carga **inswapper128.onnx** una sola vez y cambia las caras según los índices pedidos.
`core/xseg.py`	Modelo de **oclusión** (`face_occluder.onnx`): calcula qué parte de la cara es piel visible para no pintar encima de pelo, manos u objetos. Incluye la nitidez *natural* y *strong*.
`core/Face_Enhancer.py`, `face_enhancer.py`, `gpen_model.py`	Mejoradores de calidad (GPEN). En esta versión **no los usa** ni la ventana ni la CLI: son pruebas sueltas.
`cambiador.py`	Script de ejemplo independiente que cambia una cara y la mejora.
`core/xseg copy.py`, `config.py`	Una copia idéntica de `xseg.py` y un archivo vacío: restos de desarrollo.

> **Para aprender:** Un **modelo ONNX** es una red neuronal guardada en un formato estándar. `onnxruntime` es el programa que la ejecuta, en la CPU o en la GPU. TransferIa usa tres: el detector (buffalo_l), el que cambia la cara (inswapper) y el de oclusión (XSeg).

2.2 La ventana original

Así se veía el programa al abrirlo con `Transfer-IA.bat` (que solo ejecuta `python app.py`). Para la captura se cargaron dos caras **generadas por IA**: la fuente a la izquierda y el destino a la derecha.

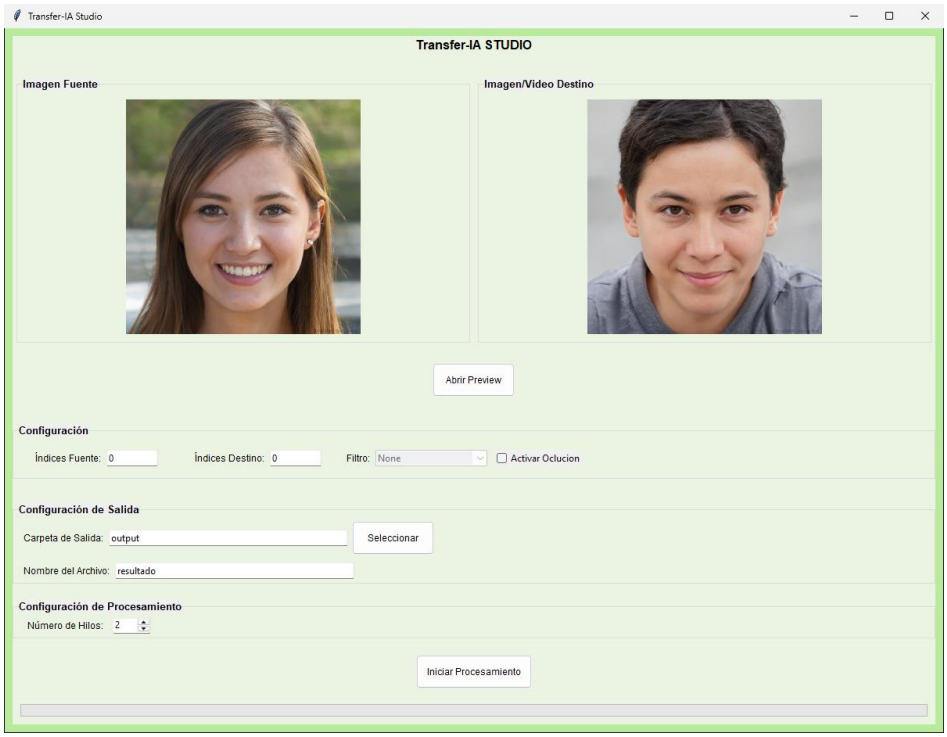


Figura 2. *Ventana original (commit `5f86333`) con las caras de prueba cargadas*

Qué hay que mirar: los dos recuadros de arrastrar y soltar, **Abrir Preview**, los **Índices Fuente/Destino** (qué cara usar si hay varias), **Activar Oclucion** con su **Filtro** de nitidez, la carpeta y el nombre de salida, y **Número de Hilos**. La ventana ya procesaba videos con varios hilos; la CLI todavía no (sección 4).

Al pulsar **Abrir Preview** se abre una ventana con el resultado:

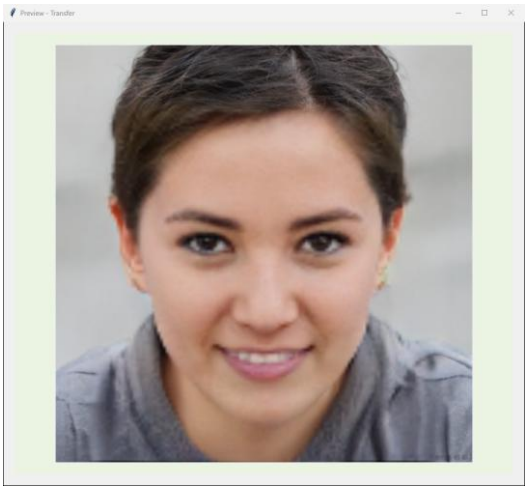


Figura 3. *Vista previa original: la identidad de la fuente sobre la postura y la ropa del destino*

2.3 La CLI original, probada

La ayuda de la CLI muestra todas las opciones. Se ejecutó la versión original tal cual:

```
> python cli.py process --help
usage: cli.py process [-h] --source SOURCE --target TARGET --output OUTPUT
                      [--models-dir MODELS_DIR] [--source-index SOURCE_INDEX]
                      [--target-index TARGET_INDEX] [--occlusion]
                      [--occlusion-mode {None,natural,strong}]
                      [--threads THREADS] [--overwrite] [--no-audio]
                      [--verbose]

options:
  -h, --help            show this help message and exit
  --source SOURCE       imagen con la cara de origen
  --target TARGET       imagen o video destino
  --output OUTPUT       archivo de salida
  --models-dir MODELS_DIR
                        carpeta de modelos; por defecto: models
  --source-index SOURCE_INDEX
                        indices de caras origen, por ejemplo: 0,1
  --target-index TARGET_INDEX
                        indices de caras destino, por ejemplo: 0,1
  --occlusion           activa el modelo de oclusion
  --occlusion-mode {None,natural,strong}
                        filtro del modelo de oclusion
  --threads THREADS     numero de hilos solicitado; por defecto: 2
  --overwrite           reemplaza el archivo de salida si ya existe
  --no-audio            no copia el audio original del video
  --verbose             muestra informacion detallada
```

Figura 4. *Ayuda de `cli.py process` en la versión original*

Y se cambió la cara de la imagen de prueba con oclusión natural. Tardó **23,5 s** en total, de los cuales unos 20 s son cargar los modelos (pasa una sola vez por ejecución):

```
python cli.py process --source fuente.jpg --target destino.jpg --output salida/e1_imagen.jpg --occlusion --
occlusion-mode natural
```



Figura 5. *Fuente, destino y resultado de la CLI original*

3. El problema: procesar un video sin saber cuánto falta

En la versión original, al procesar un video por consola **no se mostraba nada** hasta el final. Si el video era largo, no había forma de saber si el programa avanzaba o se había colgado.

Cómo reproducirlo

1. Usa la versión `5f86333` y un video cualquiera (aquí, uno de 20 fotogramas hecho con una cara de IA).
2. Ejecuta `python cli.py process --source fuente.jpg --target video_prueba.mp4 --output salida/e1_antes.mp4 --no-audio`.

3. Verás la consola en silencio hasta que aparece la última línea:

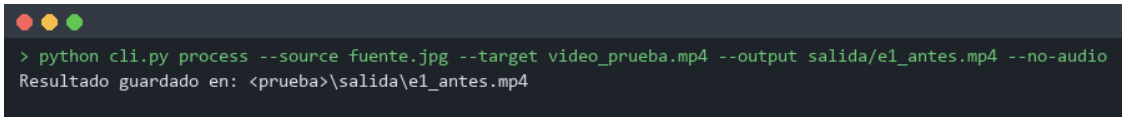


Figura 6. *Versión original: ninguna señal de avance durante el proceso*

Había una salida de emergencia, `--verbose`, que imprimía una línea por fotograma ("Procesando frame 1", "Procesando frame 2"…). En un video de 3 minutos a 30 fps son **5400 líneas**: se sabe que avanza, pero no cuánto falta.

Además, la opción `--threads` ya existía en la CLI, pero **solo se validaba** (`--threads debe ser mayor o igual que 1`): ningún código la usaba. Los fotogramas se procesaban uno detrás de otro.

4. La solución: barra de progreso y varios hilos

4.1 La barra de progreso

Qué: una línea que se reescribe sola con el porcentaje. **Por qué así:** una barra en una sola línea no llena la consola y se entiende de un vistazo. **Cómo:** el truco es el carácter `\r` (retorno de carro), que lleva el cursor al principio de la línea **sin bajar** a la siguiente; lo que se imprime después tapa lo anterior.

```
PROGRESS_BAR_WIDTH = 30

def _show_progress(current, total, label="Procesando"):
    if total > 0:
        percentage = min(current / total, 1.0)
        completed_width = int(PROGRESS_BAR_WIDTH * percentage)
        bar = "#" * completed_width + "-" * (PROGRESS_BAR_WIDTH - completed_width)
        message = f"\r{label}: [{bar}] {percentage * 100:6.2f}% ({current}/{total})"
    else:
        message = f"\r{label}: {current} frames"
    print(message, end="", flush=True)
```

- `total` sale de `capture.get(cv2.CAP_PROP_FRAME_COUNT)`: el número de fotogramas que dice el archivo de video.
- `min(current / total, 1.0)` evita pasar del 100 % si el archivo informa menos fotogramas de los reales (pasa en algunos videos).
- Si el video no informa el total (`total` = 0), muestra solo cuántos van.
- `end=""` evita el salto de línea y `flush=True` obliga a que se vea **ya**, sin esperar a que se llene el búfer.

4.2 Varios hilos, pero en orden

Qué: `--threads N` ahora sí reparte los fotogramas entre N hilos. **El reto:** los hilos terminan en cualquier orden, pero el video se tiene que escribir en orden. **Cómo lo resuelve:** cada fotograma se guarda en un diccionario con su número, y solo se escribe el que toca (`frames_written`).

```
pending_frames = {}                      # número de fotograma -> trabajo en curso
max_pending_frames = max(args.threads * 2, args.threads)
with concurrent.futures.ThreadPoolExecutor(max_workers=args.threads) as executor:
    while not reading_finished or pending_frames:
        # 1. Leer y encargar fotogramas hasta tener 2 por hilo en cola
        while not reading_finished and len(pending_frames) < max_pending_frames:
            success, frame = capture.read()
            if not success:
                reading_finished = True
                break
            pending_frames[frame_number] = executor.submit(_swap_frame, swapper, source_img, frame, args)
            frame_number += 1
        # 2. Escribir SOLO el siguiente en orden (espera a que termine)
        if frames_written not in pending_frames:
            continue
        result_frame = pending_frames.pop(frames_written).result()
        writer.write(result_frame)
        frames_written += 1
        _show_progress(frames_written, total_frames)
```

Paso a paso de lo que pasa con 2 hilos:

1. Se leen 4 fotogramas (2 por hilo) y se encargan: 0, 1, 2 y 3.
2. El bucle pide el resultado del 0 con `.result()`, que **espera** a que ese hilo termine.
3. Se escribe el 0, sube la barra y se lee el fotograma 4 para mantener la cola llena.
4. Si el 1 todavía no terminó, `.result()` espera; si el 2 terminó antes, se queda guardado hasta su turno.

> **Para aprender:** Limitar la cola a 2 fotogramas por hilo es importante: sin ese límite el programa leería **todo** el video a memoria antes de terminar el primero. Es el patrón productor-consumidor con cola acotada.

5. Cómo se comprobó

Se sacó una copia de cada versión con `git archive` (sin tocar el proyecto) y se ejecutaron con caras de IA.

La barra

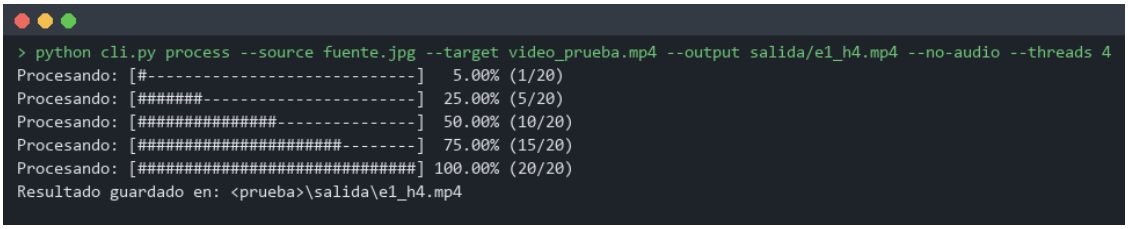


Figura 7. *Versión `1f065f1`: la barra avanza del 5 % al 100 % (aquí, algunos de sus estados)*

¿Los hilos aceleran?

Se midió el tiempo total con un video de **100 fotogramas** (480×480) en tu PC, que procesa con la CPU:

Versión	Hilos	Tiempo total
Original `5f86333`	1 (uno detrás de otro)	70,8 s
`1f065f1`	1	64,9 s
`1f065f1`	2	73,4 s
`1f065f1`	4	59,1 s

Resultado honesto: las diferencias (59 a 73 s) están dentro de la variación normal entre ejecuciones. Los hilos **casi no aceleran en CPU**.

Por qué: onnxruntime ya reparte cada cálculo de la red neuronal entre los **16 núcleos** de tu procesador. Si dos hilos piden cálculos a la vez, se reparten los mismos núcleos: es como abrir dos cajas en un supermercado que tiene un solo cajero.

> **Para aprender:** Paralelizar solo acelera si hay recursos libres. En GPU el resultado podría ser distinto (mientras un hilo usa la GPU, otro prepara el siguiente fotograma en la CPU), pero **no se probó**: este PC no tiene CUDA.

6. "mejora normal": la opción --device

El commit `85ea6f0` agregó al README la opción `--device` (auto, cuda o cpu) con este ejemplo:

```
python cli.py process --device cuda --source "src/rostro.jpg" --target "src/video.mp4" --output
"output/resultado.mp4" --threads 2
```

> **Advertencia:** `--device` **nunca se programó** en `cli.py`: no aparece en esa versión ni en las siguientes. Si se usaba, la CLI respondía `cli.py: error: unrecognized arguments: --device cuda` (comprobado). El commit del 23 de octubre la quitó del README.

Lo más probable es que la idea fuera poder exigir la GPU. Cómo se eligen GPU o CPU de verdad se explica en la entrega 2.

7. Cómo lo pruebas tú

1. Abre una consola en la carpeta de TransferIa.
2. Procesa un video corto: `python cli.py process --source src\fuente.jpg --target src\video.mp4 --output output\prueba.mp4`.
3. Verás la barra avanzar en una sola línea hasta el 100 %.
4. Si quieres comparar hilos, repite con `--threads 1` y `--threads 4` y mira el tiempo.

La función `_show_progress` sigue en la versión actual de `cli.py`, y ahora también muestra «Analizando personas» e «Imagenes». El bucle de video se reescribió el 26 de octubre (entrega 9).

8. Lo que aprendimos

- `\r` + `end=""` + `flush=True` es la receta para una barra de progreso en consola sin librerías.
- Para procesar en paralelo y escribir en orden: diccionario por número + cola acotada.
- Medir antes de concluir: los hilos parecían una mejora segura y en CPU no lo son.
- Documentar una opción no la crea: README y código tienen que ir juntos.

Glosario

Término	Significado
Fotograma (frame)	Cada imagen fija de un video. 30 fps = 30 fotogramas por segundo.
Hilo (thread)	Una línea de trabajo dentro del mismo programa. Varios hilos pueden avanzar a la vez.
ThreadPoolExecutor	Herramienta de Python que mantiene N hilos y les reparte trabajos (`submit`).
Future / .result()	El "comprobante" de un trabajo encargado; `.result()` espera a que termine y devuelve lo que produjo.
Índice de cara	Número de la cara contando de izquierda a derecha, desde 0.
Oclusión	Algo que tapa la cara (pelo, mano, micrófono). El modelo XSeg lo detecta para no pintar encima.